

Mini-projet 04 — bytes vs str : décoder des bannières

Pourquoi ce mini-projet ?

Dès que tu ouvriras une **socket** (Module D), tu ne recevras **jamais** du texte : tu recevras des **octets bruts** (**bytes**). Quand tu interrogues un service (SSH, HTTP...), il te renvoie une **bannière** — sa carte de visite — sous forme d'octets. Pour la lire, il faut la **décoder**. Et pour lui envoyer une requête, il faut **encoder** ton texte en octets. C'est la traduction texte ↔ octets, et c'est incontournable en réseau.

1. Deux types bien distincts

	str	bytes
C'est quoi	du texte (des caractères)	des octets bruts (des nombres de 0 à 255)
On écrit	"scan"	b"scan" (préfixe b)
Sert à	manipuler du texte dans ton code	écrire sur disque, envoyer sur le réseau

```
texte = "scan"      # type str
octets = b"scan"    # type bytes (regarde le petit b devant)
type(texte)         # <class 'str'>
type(octets)         # <class 'bytes'>
```

2. Traduire : encode() et decode()

str → **.encode()** → bytes bytes → **.decode()** → str

- .encode()** : texte → octets (**encoder** = mettre **en** octets pour partir sur le réseau)
- .decode()** : octets → texte (**décoder** = re-transformer en texte lisible)

```
"scan".encode()      # -> b'scan'      (str -> bytes)
b"scan".decode()     # -> 'scan'      (bytes -> str)
```

Moyen mnémo : tu **encodes** pour **envoyer**, tu **décodes** pour **déchiffrer**. Le réseau, c'est des octets ; ton écran, c'est du texte.

3. Les encodages — et pourquoi ça compte (avec le hex)

Un **encodage** est la table de correspondance « caractère → octet(s) ». Le standard aujourd'hui : **UTF-8**. Point crucial : en UTF-8, **un caractère accentué occupe plusieurs octets**.

```
# Un caractère ASCII simple = 1 octet
"A".encode("utf-8")      # -> b'A'      (1 octet : 0x41)

# Un caractère accentué = PLUSIEURS octets en UTF-8
"é".encode("utf-8")      # -> b'\xc3\xa9' (2 octets : 0xc3 0xa9)
len("é")                 # 1  <- 1 caractère
len("é".encode("utf-8")) # 2  <- mais 2 octets !
```

Décomposition de "é" en UTF-8 :

caractère é → octet 1 = **0xC3** (195) + octet 2 = **0xA9** (169)

en latin-1 (vieil encodage) : é → **0xE9** (233), un seul octet

Même caractère, octets différents selon l'encodage. D'où la règle : **on décode avec le même encodage qui a servi à encoder**.

4. ⚠ Les pièges

Piège n°1 — on ne mélange pas str et bytes.

```
"scan " + b"cible"
# TypeError: can only concatenate str (not "bytes") to str
```

Il faut d'abord tout ramener dans le même type (tout décoder, ou tout encoder) avant de coller.

Piège n°2 — décoder avec le mauvais encodage. Si des octets latin-1 sont décodés en UTF-8, soit ça plante (`UnicodeDecodeError`), soit tu obtiens du charabia (« mojibake », genre `Ã©` au lieu de `é`).

```
b'\xe9'.decode("utf-8")
# UnicodeDecodeError: 'utf-8' codec can't decode byte 0xe9...
```

Piège n°3 — la parade : le paramètre `errors=`. Sur le réseau, tu reçois parfois des octets « sales ». Pour éviter le crash :

```
b'\xe9 salut'.decode("utf-8", errors="replace")
# -> '  salut' (l'octet illisible devient  , pas de crash)
b'\xe9 salut'.decode("utf-8", errors="ignore")
# -> ' salut' (l'octet illisible est zappé)
```

5. Le contexte réseau : une bannière

Quand tu te connectes à un serveur SSH, il t'envoie sa bannière, en octets, terminée par un retour chariot `\r\n` :

```
brut = b"SSH-2.0-OpenSSH_9.6\r\n" # ce que la socket te donne
brut.decode().strip()           # -> 'SSH-2.0-OpenSSH_9.6' (texte propre)
```

Note : `\r\n` = retour chariot + saut de ligne (fin de ligne réseau, style Windows). Un simple `.strip()` les enlève, comme les espaces.

🎯 Ta mission

Objectif : écrire les deux fonctions de traduction que tout code réseau utilise.

- `decoder_banniere(brut)` → prend des **bytes**, renvoie une **str** propre (décodée en UTF-8, nettoyée avec `strip()`, et **sans crash** si un octet est illisible → utilise `errors="replace"`).
- `encoder_requete(texte)` → prend une **str**, renvoie des **bytes** prêts à envoyer sur une socket.

Tests à faire passer :

```
print(decoder_banniere(b"SSH-2.0-OpenSSH_9.6\r\n"))
# -> SSH-2.0-OpenSSH_9.6

print(decoder_banniere(b"HTTP/1.1 200 OK\r\n"))
# -> HTTP/1.1 200 OK

print(decoder_banniere(b"\xe9 serveur bizarre")) # octet sale
# -> ne doit PAS planter (tu verras un caractère de remplacement)

r = encoder_requete("GET / HTTP/1.0")
print(r) # -> b'GET / HTTP/1.0'
print(type(r)) # ->
```

Bonus :

- affiche la bannière SSH **octet par octet en hexadécimal** — indice : `for octet in brut:` te donne des **entiers** (0-255), et `f"{octet:02x}"` les affiche en hex sur 2 chiffres.
- compare `len("é")` et `len("é".encode())` et explique en commentaire pourquoi c'est différent.

✓ **Critères de réussite**

- `decoder_banniere` renvoie bien une **str** nettoyée.
- Elle ne plante pas sur `b"\xe9 serveur bizarre"`.
- `encoder_requete` renvoie bien un objet de type **bytes**.

Si tu bloques — indices progressifs

Étape	Indice
Décoder	<code>brut.decode("utf-8", errors="replace")</code> puis <code>.strip()</code> . Tu peux enchaîner : <code>brut.decode(...).strip()</code> .
Ne pas planter	C'est <code>errors="replace"</code> qui empêche le crash sur un octet illisible.
Encoder	<code>texte.encode("utf-8")</code> suffit. La fonction fait <code>return</code> le résultat.
Bonus hex	<code>" ".join(f"{o:02x}" for o in brut)</code> → une chaîne du genre <code>53 53 48 ...</code> .

Où ranger : `mini-projets/04-bytes-vs-str/banniere.py` .
Quand c'est prêt : `git add .` → `commit` → `push` , puis « c'est poussé ».